B.TECH CCE REVISION · PART 2

Networks and the Web

Covers the marksheet subjects Computer Network Protocols, Principles of Data Communication, Network Programming, Advanced Communication Networks, and Wireless Communication.

How data actually moves: layers, addresses, TCP and UDP, DNS, HTTP and TLS, and the plumbing (load balancers, CDNs, proxies) that every web, cloud and AI product runs on.

Isagi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

How to use this chapter

Networking was the weakest area on the revision map, and it is also the one everything else sits on. A cloud deployment, an API call to an LLM, an agent calling a tool and a voice upload from a phone are all network events first. When they break, the symptoms (timeouts, slow first loads, "connection reset", certificate errors) are network symptoms.

The target is not exam fluency. It is to be able to explain each idea below to a non-technical person in a minute or two, and to reason about it when something breaks.

> **How to read it.** Read one section, then do its lab in Section 12. Each lab takes 15–40 minutes with nothing but a laptop. Section 13 has self-check questions with answers. If you can answer them without looking, the module is done.

Contents

1. The big picture: what a network is for
2. Layers, and why they exist
3. Data communication: signals, bandwidth, latency
4. The link layer: Ethernet, Wi-Fi, switches, MAC addresses
5. IP: addresses, subnets, NAT, routing
6. Transport: TCP, UDP and QUIC
7. DNS
8. HTTP, from 1.1 to 3
9. TLS and certificates
10. Network programming: sockets, servers, WebSockets, streaming
11. The plumbing: load balancers, CDNs, proxies, rate limits, wireless and cellular
12. Labs, and a troubleshooting playbook
13. Self-check questions and answers
14. Resources

1. The big picture

A network moves bytes from one program to another program on a different machine, reliably enough and fast enough.

Everything in this chapter is a solution to a sub-problem of that sentence.

- **Finding the other machine.** Names (DNS) map to addresses (IP), and routers forward packets hop by hop toward that address.
- **Sharing the wire.** Many conversations share the same cables and radio channels, so data is chopped into **packets** that are interleaved. This is *packet switching*. The old telephone network reserved a whole circuit per call instead (*circuit switching*).
- **Coping with loss.** Packets get dropped, duplicated and reordered. Something has to detect that and recover (TCP), or the application decides it doesn't care (UDP for live voice or games).
- **Not overwhelming anyone.** The receiver can be slower than the sender (*flow control*), and the network in between can be congested (*congestion control*).
- **Keeping it private and authentic.** Anyone on the path can read or alter unencrypted traffic. TLS fixes that.

Two numbers describe any network path, and most performance reasoning comes down to them:

Measure	What it means	Typical values
Bandwidth (throughput)	How many bits per second the pipe carries	Home fibre 100–1000 Mbps; 4G 10–50 Mbps; data-centre links 10–400 Gbps
Latency (round-trip time, RTT)	How long a packet takes to get there and a reply to come back	Same city 5–20 ms; Mumbai–Singapore ~60 ms; India–US East ~200–250 ms; mobile adds 20–80 ms

Bandwidth is like the width of a highway; latency is its length. For most web and API work, **latency dominates**, because a request involves several round trips before any useful data arrives. Upgrading from 100 to 1000 Mbps does almost nothing for a 5 KB API call to a server 250 ms away.

> **The speed of light is a hard floor.** Light in fibre travels about 200,000 km/s, roughly 200 km per millisecond. Mumbai to Virginia is about 13,000 km of fibre, so ~65 ms one way and ~130 ms round trip before any router or server does anything. No engineering removes that; you can only move the server closer (CDNs, regional deployments) or need fewer round trips (HTTP/2, QUIC, connection reuse).

2. Layers, and why they exist

Each layer solves one problem and trusts the layer below for everything else.

Networking is split into layers so that each part can change independently. Your browser doesn't care whether you're on Wi-Fi or fibre; Wi-Fi doesn't care whether you're loading a web page or a video call. The practical model has four or five layers (the seven-layer OSI model is the textbook version; layers 5 and 6 rarely matter in practice).

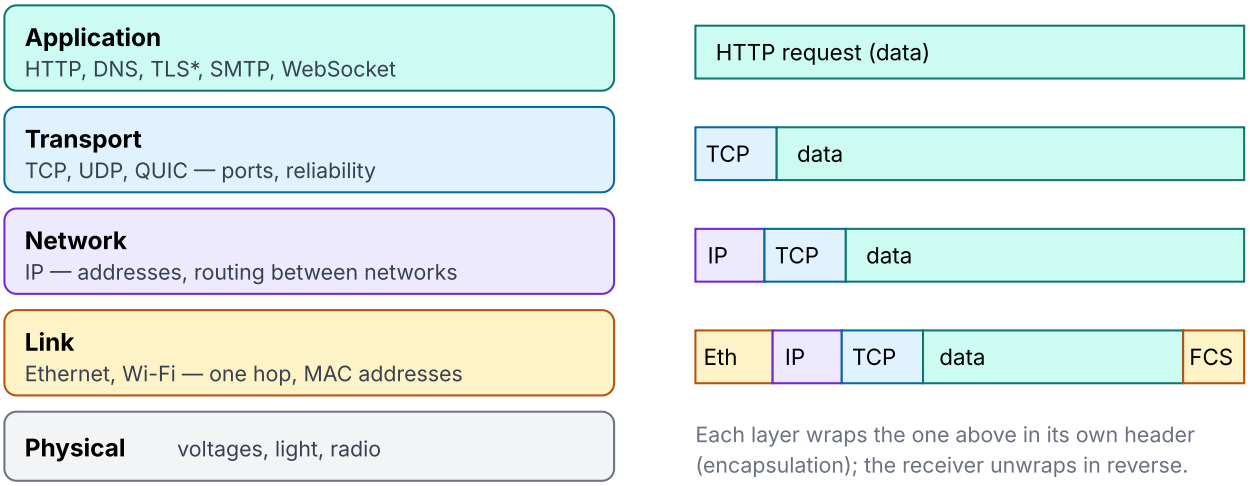


Figure 1. The practical layer model and encapsulation. *TLS sits between TCP and HTTP.

Layer	Question it answers	Address it uses	Device that works at it
Application	What does the program want to say?	URLs, hostnames	Your app, web server
Transport	Which program on the machine, and did it all arrive?	Port number (443, 5432)	Operating system
Network	Which machine, across the whole internet?	IP address	Router
Link	Which device on this local segment?	MAC address	Switch, Wi-Fi access point
Physical	How are bits turned into signals?	—	Cables, radios, NIC

The key idea is **encapsulation**: each layer adds its own header on the way down and strips it on the way up. A router only reads up to the IP header; a switch only reads the Ethernet header. That's why the internet can carry protocols invented decades after the routers were built.

A useful mental shortcut: **IP gets a packet to the right machine; the port gets it to the right program; TCP makes sure it all arrives in order.**

3. Data communication

How bits become signals, and the limits that physics puts on them.

This was "Principles of Data Communication". Only a handful of ideas matter outside exams.

- **Analogue vs digital.** Bits are sent by varying a physical signal: voltage on copper, light pulses in fibre, radio waves in Wi-Fi and cellular. **Modulation** is how bits are encoded in a carrier wave (changing its amplitude, frequency or phase). Modern Wi-Fi and 4G/5G use **QAM**, which encodes several bits per symbol by varying both amplitude and phase; better signal lets the radio pack more bits per symbol, which is why speed drops as you walk away from the router.
- **Bandwidth in the original sense** is the range of frequencies a channel uses (in hertz). More hertz means more possible data rate. That's why spectrum auctions are worth billions.
- **Shannon's limit**: the maximum error-free data rate of a channel is $C = B \times log_2(1 + S/N)$, where B is bandwidth in hertz and S/N is signal-to-noise ratio. Plain meaning: capacity grows linearly with spectrum but only logarithmically with signal strength. Doubling transmit power buys much less than doubling spectrum.
- **Errors happen.** Noise flips bits. Links detect this with checksums or CRCs (the "FCS" at the end of an Ethernet frame) and drop bad frames. Wireless links also use **forward error correction**: extra redundant bits that let the receiver fix some errors without asking again.
- **Multiplexing** lets many conversations share one medium: by time slots (TDM), by frequency (FDM), by codes (CDMA in 3G) or by many narrow sub-carriers (OFDM in Wi-Fi, 4G and 5G).
- **Delays** add up from four sources: *transmission* (pushing the bits onto the wire, = size ÷ rate), *propagation* (distance ÷ signal speed), *queuing* (waiting in a router's buffer) and *processing*. On long paths propagation dominates; on congested paths queuing does.

Worked example: why a big upload on a slow link feels sluggish

A 2 MB voice recording uploaded over a 4G uplink at 5 Mbps: transmission time = 2 × 8 Mb ÷ 5 Mbps = **3.2 seconds**, regardless of where the server is. Over Wi-Fi at 50 Mbps uplink: 0.32 s. Here bandwidth, not latency, dominates, because the payload is big. Compressing the audio (Opus at 32 kbps instead of WAV at ~1.4 Mbps) is a 40× win and far cheaper than any network fix.

4. The link layer

Getting a frame across one hop: from your laptop to the router, or between two machines on the same switch.

- **MAC address**: a 48-bit hardware address burned into each network interface (`3c:22:fb:9a:10:4e`). It only matters on the local segment; it never crosses a router.
- **Ethernet** is wired LAN. A **switch** learns which MAC address is on which port and forwards frames only to the right port. (Old **hubs** sent everything everywhere.)
- **ARP** (Address Resolution Protocol) answers "which MAC address has IP 192.168.1.1?" on the local network. Your laptop broadcasts the question; the router replies. Spoofing ARP replies is a classic way to intercept traffic on a shared network, which is one reason public Wi-Fi is untrusted.
- **Wi-Fi** (802.11) is the same idea over radio, with extra problems: devices can't hear each other's collisions, signals fade through walls, and everyone shares the channel. Wi-Fi therefore takes turns (CSMA/CA), acknowledges every frame, and retransmits a lot. A "full-bar" Wi-Fi connection can still have high and jittery latency when the channel is busy.
- **VLANs** split one physical switch into several logical networks, for example to keep office guests away from point-of-sale machines.
- **MTU** (maximum transmission unit) is the largest frame payload a link carries, usually 1500 bytes for Ethernet. Bigger IP packets get fragmented or dropped. VPNs and tunnels add headers and shrink the effective MTU, which causes the classic "small requests work, big ones hang" bug.

5. IP: addresses, subnets, NAT, routing

The internet's job is to get a packet to the right machine anywhere in the world, one hop at a time.

Addresses

An **IPv4 address** is 32 bits written as four numbers: `203.0.113.25`. There are only about 4.3 billion, which ran out years ago. **IPv6** uses 128 bits (`2001:db8::25`) and is widely deployed in India: Jio's mobile network is largely IPv6.

Subnets and CIDR

An address is split into a **network part** and a **host part**. CIDR notation says how many leading bits are the network: `192.168.1.0/24` means the first 24 bits are fixed, leaving 8 bits (256 addresses, 254 usable) for hosts.

CIDR	Addresses	Typical use
/32	1	A single host in a firewall rule
/28	16	A tiny cloud subnet
/24	256	A home or office LAN
/16	65,536	A whole cloud VPC
/0	everything	"Any address" in a route or rule

Private ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) are never routed on the public internet. Your laptop at home and a cloud server in a VPC both live in private ranges.

Worked example: is this address in my subnet?

Is `10.0.5.200` inside `10.0.4.0/23`? A /23 fixes the first 23 bits, so the third number moves in steps of 2 (2^(24−23)). The block starts at `10.0.4.0` and covers `10.0.4.0` – `10.0.5.255`. **Yes.** `10.0.6.1` would not be. Cloud security groups, VPC peering and firewall rules are full of this arithmetic. Python's `ipaddress` module does it for you (Lab 4).

NAT

Your home router has one public IP from the ISP; every device behind it has a private IP. **NAT** (network address translation) rewrites outgoing packets to use the public IP and a unique port, and remembers the mapping so replies come back to the right device. Consequences:

- Devices behind NAT can make outgoing connections easily, but nobody outside can start a connection *to* them. That's why phone apps poll or keep a connection open to a server instead of being "called" directly, and why peer-to-peer video calls need tricks (STUN/TURN in WebRTC).
- Mobile carriers use **carrier-grade NAT**: thousands of customers share one public IP. Rate-limiting by IP address can therefore block a whole neighbourhood.

Routing

Each **router** holds a table: "for destinations matching this prefix, send to that next hop". It picks the **longest matching prefix**. Inside an organisation, routing protocols (OSPF) share topology; between organisations (ISPs, clouds, Jio, Airtel), **BGP** exchanges which networks each can reach. BGP is policy-driven and trust-based, so a misconfiguration at one big network can take down large parts of the internet (Facebook's 2021 outage was a BGP withdrawal).

ICMP carries error and diagnostic messages. `ping` uses it to measure round-trip time; `traceroute` exploits the packet TTL (time-to-live, decremented at each hop) to make each router along the path reveal itself.

6. Transport: TCP, UDP and QUIC

IP delivers packets to a machine, best-effort. Transport turns that into a conversation between two programs.

A connection is identified by five things: source IP, source port, destination IP, destination port, protocol. Servers listen on well-known ports (443 for HTTPS, 5432 for Postgres, 22 for SSH); clients use random high ports.

TCP: a reliable, ordered byte stream

- **Three-way handshake** (SYN, SYN-ACK, ACK) before any data: costs one round trip.
- **Sequence numbers and acknowledgements**: every byte is numbered; the receiver acknowledges what it got; anything not acknowledged in time is resent.
- **Flow control**: the receiver advertises a window ("I can take 64 KB more") so a fast sender can't overflow a slow receiver.
- **Congestion control**: TCP starts slowly (**slow start**), doubling its sending rate every round trip until it sees loss, then backs off. Algorithms like CUBIC and BBR tune this.
- **Head-of-line blocking**: because TCP delivers bytes strictly in order, one lost packet stalls everything behind it until it's resent, even data that has already arrived.

UDP: just send it

No handshake, no retransmission, no ordering. Each datagram stands alone. That sounds worse, but it's right when **late data is useless**: live voice, video calls, games, DNS lookups (a single small question and answer). The application handles loss itself, for example by skipping a 20 ms audio frame rather than freezing the call.

QUIC: TCP's job, rebuilt on UDP

QUIC (the basis of HTTP/3) runs over UDP and reimplements reliability and congestion control in user space, with TLS 1.3 built in.

	TCP + TLS 1.3	QUIC
Round trips before first request	2 (TCP + TLS)	1 (0 when resuming)
Lost packet blocks other streams?	Yes	No, streams are independent
Survives Wi-Fi → 4G switch?	No, the IP changed so the connection dies	Yes, connections have IDs, not just IP/port
Where it's implemented	OS kernel	Application/library, so it evolves faster

Worked example: why a fast link can still be slow

A new TCP connection starts with a congestion window of about 10 packets (~14 KB). Each round trip, it roughly doubles: 14, 28, 56, 112 KB… Sending a 1 MB response therefore takes about 6–7 round trips even on gigabit fibre. At 250 ms RTT (India to US), that's over 1.5 seconds of pure ramp-up, before counting the TCP and TLS handshakes. Reusing connections (keep-alive, HTTP/2) and putting servers closer to users matter far more than raw bandwidth.

The other classic formula is the **bandwidth-delay product**: to keep a pipe full you need *bandwidth × RTT* bytes in flight. A 100 Mbps path at 200 ms needs 2.5 MB in flight; if the receive window is only 64 KB, throughput is capped at 64 KB ÷ 0.2 s ≈ 2.6 Mbps, whatever the link speed.

7. DNS

The internet's phone book: it turns `api.example.com` into an IP address, and it is involved in a surprising share of outages.

When your laptop needs an address:

1. It checks its own cache, then asks a **recursive resolver** (your ISP's, or a public one like 1.1.1.1 or 8.8.8.8).
2. If the resolver doesn't have it cached, it asks a **root server** ("who handles `.com`?"), then the **TLD server** ("who handles `example.com`?"), then the domain's **authoritative server**, which gives the answer.
3. The answer comes back with a **TTL** (time to live), so everyone caches it for that long.

Record	Holds	Example use
A / AAAA	IPv4 / IPv6 address	`api.example.com → 203.0.113.25`
CNAME	Alias to another name	`www → example.com`, or pointing at a CDN or Vercel
MX	Mail server for the domain	Google Workspace email
TXT	Free text	Domain verification, SPF/DKIM email anti-spoofing
NS	Which servers are authoritative	Delegating the domain to Cloudflare

Things that bite in practice:

- **Propagation is just caching.** After you change a record, old answers live on until their TTL expires. Lower the TTL a day *before* a migration.
- **DNS is usually UDP and unencrypted**, so networks can see and tamper with it. DNS-over-HTTPS (DoH) fixes that.
- **"It's always DNS."** Expired domains, wrong CNAMEs, missing records after a migration and resolver outages cause a large fraction of "the site is down" incidents.
- DNS is also a crude load balancer and geo-router: the same name can return different IPs by location (this is how CDNs send you to a nearby server).

8. HTTP

The application protocol of the web and of almost every API, including LLM APIs.

An HTTP request is a method, a path, headers and an optional body; the response is a status code, headers and a body.

```
POST /v1/messages HTTP/1.1
Host: api.example.com
Authorization: Bearer sk-...
Content-Type: application/json

{"model": "...", "messages": [{"role": "user", "content": "Hi"}]}

HTTP/1.1 200 OK
Content-Type: application/json

{"content": [{"type": "text", "text": "Hello!"}]}
```

Methods	Meaning	Safe to retry?
GET	Read	Yes
PUT / DELETE	Replace / remove (same result if repeated)	Yes (idempotent)
POST	Create or "do something"	Not by default; use an idempotency key
PATCH	Partial update	Depends

Status class	Means	Common ones
2xx	It worked	200 OK, 201 Created, 204 No Content
3xx	Look elsewhere	301/308 moved permanently, 304 not modified (use your cache)
4xx	Your request is wrong	400 bad request, 401 not authenticated, 403 not allowed, 404 not found, 409 conflict, 429 too many requests
5xx	The server failed	500 internal error, 502 bad gateway, 503 unavailable, 504 gateway timeout

A useful habit: **4xx means fix the request, 5xx means retry later (with backoff), 429 means slow down.**

- **Headers** carry metadata: `Content-Type`, `Authorization`, `Cache-Control`, `Accept-Encoding: gzip`, `User-Agent`.
- **Cookies** are headers the server sets (`Set-Cookie`) and the browser sends back automatically. That's how logins persist, and why CSRF attacks work (Part 6). Flags `HttpOnly`, `Secure` and `SameSite` limit the damage.
- **HTTP is stateless**: each request stands alone; sessions are layered on with cookies or tokens.
- **REST** is a style: resources as URLs (`/sessions/42/scores`), standard methods, JSON bodies. **GraphQL** lets the client ask for exactly the fields it wants through one endpoint. **gRPC** uses HTTP/2 and binary messages between services.
- **Caching**: `Cache-Control: max-age=3600` lets browsers and CDNs reuse a response. `ETag` lets a client ask "has this changed?" and get a cheap 304.

Versions

- **HTTP/1.1**: text, one request at a time per connection; browsers open ~6 connections per site to compensate.

- **HTTP/2**: binary, many requests **multiplexed** over one TCP connection, header compression. Still suffers TCP head-of-line blocking.
- **HTTP/3**: HTTP/2's ideas over QUIC; better on lossy mobile networks.

9. TLS and certificates

TLS makes a connection private (encrypted), intact (tamper-evident) and authentic (you're talking to the real server). The "S" in HTTPS.

The TLS 1.3 handshake, simplified:

1. The client says hello, listing supported ciphers, and sends a key share.
2. The server replies with its key share and its **certificate**. Both sides now derive the same secret session key (Diffie-Hellman key exchange) without ever sending it.
3. The client checks the certificate: is it for this hostname, unexpired, and signed by a **certificate authority (CA)** the device trusts?
4. Everything after that is encrypted with fast symmetric encryption (AES or ChaCha20).

The certificate chain is the trust mechanism: your OS ships with ~150 trusted root CAs; a root signs an intermediate, which signs the site's certificate. **Let's Encrypt** made certificates free and automatic, which is why nearly all sites are HTTPS now.

What TLS does and doesn't hide: an observer still sees which IP you connect to, roughly how much data flows and (usually) the hostname, via the SNI field. They don't see the path, headers or body.

Common TLS errors and what they mean:

Error	Usual cause
Certificate expired	Auto-renewal broke
Hostname mismatch	Cert is for `example.com`, you hit `api.example.com`
Unknown issuer / self-signed	Internal or dev cert, or a corporate proxy intercepting traffic
Handshake failure	Old client or server, no shared TLS version or cipher

The crypto underneath (symmetric vs asymmetric keys, signatures, hashing) is covered properly in Part 6.

10. Network programming

How programs actually use the network: sockets, servers, and the patterns built on them.

A **socket** is the OS's handle for one end of a connection. A server *binds* to a port, *listens*, and *accepts* connections; a client *connects*. Then both *send* and *receive* bytes. Every web framework, database driver and HTTP library is built on this.

```
# A minimal TCP echo server using asyncio (one process, many clients)
import asyncio

async def handle(reader, writer):
    peer = writer.get_extra_info("peername")
    while data := await reader.read(1024):
        writer.write(data)          # echo back
        await writer.drain()
    writer.close()
    print("closed", peer)

async def main():
    server = await asyncio.start_server(handle, "127.0.0.1", 9000)
    async with server:
        await server.serve_forever()

asyncio.run(main())
```

Ideas worth holding onto:

- **TCP is a byte stream, not a message stream.** One `send` of 3 KB can arrive as two reads, and two small sends can arrive as one. Protocols add **framing**: a length prefix, a delimiter like newline, or HTTP's `Content-Length`.
- **Blocking vs non-blocking.** A naive server handles one client at a time. Real servers use threads, processes or an **event loop** (async I/O) to juggle thousands of connections. Part 3 explains the OS side.
- **Timeouts everywhere.** A connection can hang forever if the other side vanishes. Always set connect and read timeouts; the defaults are often infinite.
- **Keep-alive and pooling.** Opening a connection costs round trips (TCP + TLS), so clients reuse them. Database drivers and HTTP clients keep **connection pools**.
- **Polling vs push.** To get updates, a client can poll ("anything new?") every few seconds, use **long-polling**, use **Server-Sent Events** (the server streams events down one HTTP response; this is how LLM APIs stream tokens), or open a **WebSocket** (a full two-way channel upgraded from HTTP, used for chat and live audio).
- **Streaming matters for AI products.** Streaming tokens or audio means the user sees the first output after ~0.5 s instead of waiting 10 s for the whole thing. It's the single biggest perceived-speed improvement for LLM and voice features.

11. The plumbing

What sits between users and your code in any real deployment.

Component	What it does	Why you'd care
Reverse proxy (Nginx, Envoy)	Receives requests on behalf of servers; terminates TLS, compresses, routes paths	One public entry point; servers stay private
Load balancer	Spreads requests across many servers; drops unhealthy ones via health checks	Scale and survive a server dying. L4 balances TCP connections; L7 understands HTTP
CDN (Cloudflare, CloudFront, Akamai)	Caches content in hundreds of cities near users	Static files and media load in tens of ms instead of hundreds; absorbs DDoS
API gateway	Auth, rate limiting, request logging in front of APIs	Policy in one place
Forward proxy / VPN	Routes your outbound traffic through another point	Corporate control, privacy, reaching private networks
NAT gateway	Lets private cloud servers make outbound calls	Your private backend can call an LLM API
Firewall / security group	Allows or blocks by IP, port, protocol	Only 443 open to the world; DB reachable only from the app

Rate limiting protects services: e.g. 60 requests per minute per API key, implemented with token buckets. When you hit someone else's limit you get 429 with a `Retry-After` header. The correct client response is **exponential backoff with jitter**: wait 1 s, 2 s, 4 s… plus randomness so that thousands of clients don't retry in lockstep.

Wireless and cellular, conceptually

- **Wi-Fi** covers tens of metres, shares the channel among everyone nearby, and has latency that spikes when it's busy. 2.4 GHz travels further through walls; 5 and 6 GHz are faster but shorter-range.
- **Cellular (4G LTE, 5G)**: your phone talks to a base station (cell tower), which connects through the operator's core network to the internet. Radios sleep to save battery, so the first packet after idle can take 50–200 ms extra to "wake" the connection. Handovers between towers and between Wi-Fi and cellular break TCP connections (QUIC survives them).
- **5G** brings more spectrum (so more bandwidth) and lower radio latency, but real-world latency to a distant server is still dominated by distance and the core network.
- For a product used by students on budget Android phones and patchy 4G, assume: high and variable latency, uploads slower than downloads, connections that drop mid-request. Design for resumable uploads, retries, small payloads and offline tolerance.

12. Labs and a troubleshooting playbook

Each lab is 15–40 minutes. On Windows, use WSL or Git Bash for the command-line ones.

Lab 1. Watch a real request. Run `curl -v https://pipsy.space -o /dev/null`. Identify in the output: the IP address it connected to, the TLS version, the certificate's issuer and expiry, the request headers, the status code and the response headers. Then add `-w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} first_byte:%{time_starttransfer} total:%{time_total}\n"` and read where the time goes.

Lab 2. DNS by hand. `dig pipsy.space`, `dig pipsy.space NS`, `dig +trace pipsy.space`. Find the TTL, the authoritative servers, and the root → TLD → authoritative chain. Compare `dig @1.1.1.1` and `dig @8.8.8.8`.

Lab 3. Browser network tab. Open any heavy site with DevTools → Network, disable cache, reload. Sort by time. Find: the HTML request, the largest file, anything served from a CDN, the protocol column (h2/h3), and the waterfall's DNS/connect/TLS/waiting phases. Then throttle to "Slow 4G" and reload.

Lab 4. Subnet arithmetic in Python.

```
import ipaddress
net = ipaddress.ip_network("10.0.4.0/23")
for ip in ["10.0.5.200", "10.0.6.1"]:
    print(ip, ipaddress.ip_address(ip) in net)
print(net.num_addresses, net[0], net[-1])
```

Lab 5. Echo server. Run the server in Section 10, connect with `nc 127.0.0.1 9000` from two terminals at once, type, and watch both get echoed. Then kill the client abruptly and see the server notice.

Lab 6. Path and latency. `ping` a server in India, Singapore and the US (pick any well-known site hosted there); `traceroute` (Windows: `tracert`) to the farthest. Relate the RTTs to the speed-of-light estimate in Section 1.

Lab 7. Trace an audio upload. In any app you're building (or a sample upload form), upload a 1–2 MB file with DevTools open. Note the request method, `Content-Type` (multipart or direct), how long transmission took vs waiting for the server, and what happens if you switch to "Offline" mid-upload. Write two sentences on what you'd change for users on weak 4G.

Troubleshooting playbook

Work bottom-up and split the problem in half at each step.

Symptom	First checks	Likely culprit
"Could not resolve host"	`dig name`; try another resolver	DNS record missing or wrong; expired domain
Connection timeout	`ping` IP, `nc -vz host 443`	Firewall/security group blocks port; server down; wrong IP
Connection refused	Is the process listening? (`ss -ltnp`)	Nothing on that port; app crashed; wrong port
TLS / certificate error	`curl -v`, check expiry and hostname	Expired cert, wrong name, proxy interception
502 / 504 from proxy or LB	Proxy logs; can the proxy reach the backend?	Backend crashed, slow or unreachable
429	Response headers	You hit a rate limit; back off
Works on Wi-Fi, fails on mobile	MTU, IPv6, carrier NAT	IPv6-only path issue, blocked port, large packets dropped
Slow first load, fast after	Waterfall: DNS, connect, TLS times	Cold connections, far server, no CDN
Slow everything	Payload sizes, RTT, server time (TTFB)	Server processing or huge responses, not "the network"

13. Self-check questions

1. **What happens between typing a URL and seeing the page?** DNS resolves the name to an IP; the browser opens a TCP connection (or QUIC); TLS handshake verifies the certificate and sets up keys; the browser sends an HTTP GET; the server responds with HTML; the browser parses it and fetches CSS, JS and images (often over the same connection), then renders.
2. **Why does a gigabit connection not make an API call to the US faster?** Small requests are latency-bound: several round trips at ~250 ms each. Bandwidth only helps once lots of data is flowing.
3. **When would you choose UDP over TCP?** When late data is worthless (live voice, games, video calls) or the exchange is a single small message (DNS). The app handles loss itself.
4. **What does a /24 mean?** The first 24 bits are the network; 256 addresses in the block.
5. **Why can't a server just call your phone directly?** The phone is behind NAT (often carrier-grade); inbound connections have no mapping. So phones keep connections open to servers or use push services.
6. **What's head-of-line blocking?** In TCP, one lost packet holds up all later data until it's resent. HTTP/2 over TCP suffers it across all streams; QUIC avoids it.
7. **You changed a DNS record an hour ago and some users still hit the old server. Why?** Caches hold the old answer until its TTL expires.
8. **401 vs 403?** 401: we don't know who you are (log in). 403: we know who you are and you're not allowed.
9. **What does a certificate prove?** That a CA your device trusts vouched that the holder of this key controls this hostname.
10. **What does TLS not hide?** Which server IP you talk to, timing and volume, and usually the hostname (SNI).
11. **Why is "TCP is a byte stream" a bug source?** Message boundaries aren't preserved; code must frame messages (length prefix, delimiter) or it will mis-parse split or merged reads.
12. **What's a CDN for?** Serving cached content from near the user to cut latency and offload the origin; also absorbs attacks.
13. **How should a client handle a 503 or 429?** Retry with exponential backoff and jitter, honour `Retry-After`, and only retry idempotent operations (or use idempotency keys).
14. **Why do LLM APIs stream with Server-Sent Events?** So the user sees tokens as they're generated instead of waiting for the full response; SSE is simple one-way streaming over HTTP.
15. **Why does QUIC survive switching from Wi-Fi to 4G?** Connections are identified by connection IDs, not the IP/port 4-tuple, so a new IP doesn't kill them.
16. **What's the bandwidth-delay product, and why care?** Bandwidth × RTT = bytes needed in flight to fill the pipe; small windows cap throughput on long paths regardless of link speed.

14. Resources

Check these are still available before committing time.

- **Kurose and Ross, *Computer Networking: A Top-Down Approach*** and the authors' free lecture videos. The standard text; read chapters 1–3 and skim 4–6.
- **Ilya Grigorik, *High Performance Browser Networking*** (free online at hpbn.co). The best single source on latency, TCP, TLS, HTTP/2, mobile networks. Some parts predate HTTP/3.
- **Beej's Guide to Network Programming** (free). Sockets from first principles in C; read for the concepts.
- **Julia Evans's networking zines and blog** (jvns.ca). Short, visual, practical; excellent on DNS and HTTP.
- **Cloudflare Learning Center**. Plain-language pages on DNS, CDNs, DDoS, TLS.
- **Wireshark** and its sample captures, once you want to see packets for real.
- **Cisco Packet Tracer**, to build and simulate small networks with switches, routers and subnets.